ECEN 3753 Labyrinth Final Project Report

Leo Marcinkus

1. Final Functionality Summary

The Labyrinth project is complete. The game generates a random 15x15 maze with walls, holes, and waypoints on every reset, using gyro as an RNG seed to guarantee a different layout each time. The drone responds to board tilt through a full physics simulation with gravity projected onto X/Y, velocity integration, and wall collision. The drone must navigate three target waypoints in order within 30 seconds to win. The disruptor button allows the drone to pass through walls and holes while active. On deactivation it steps the drone forward along its path until it clears all obstacles.

All six RTOS tasks run under FreeRTOS with correct mutex-protected shared data, semaphore-driven button handling, and event-flag communication between physics and game logic. The LCD renders the full maze at startup and updates only the drone dot. The green LED tracks energy level through software PWM. The red LED flashes at a rate proportional to remaining charge time below MinActivationEnergy. All three loss conditions: hole fall-in, excessive tilt, and a time run out are detected and displayed on a game-over screen. A green win screen shows time elapsed as the score.

2. Final Summary Effort and Estimate Numbers

I have completed 100% of my estimated work (42.5 estimated hours for all completed work) in approximately 106% of the budgeted total-project time (45.0 hours actual spent, against 42.5 hours estimated). For all completed work, I took 1.06x as much time as I estimated.

The original project estimate was 35.0 hours. The final estimate grew to 42.5 hours as weekly documentation rows (0.5 hr each) were added across weeks 4 and 5. The project finished slightly over budget, driven primarily by the disruptor task which ran 3.5 hours over estimate due to a hard fault that required significant debugging time.

Task	Est. (hr)	Actual (hr)	Status
Week 1 planning documentation	2.0	2.5	Complete
Project setup	0.5	0.5	Complete
Config data structure	1.5	1.0	Complete
RTOS task structure	2.5	1.5	Complete
Gyro integration (GyroTask)	2.0	0.5	Complete
Physics helpers	1.5	0.5	Complete
Physics engine (PhysicsTask)	3.5	4.0	Complete
Green LED PWM	1.5	1.5	Complete
Red LED flash rate	2.0	1.5	Complete

Task	Est. (hr)	Actual (hr)	Status
LCD display	2.5	3.0	Complete
Hole fall-in detection	1.0	0.5	Complete
Maze generation (PRNG)	3.0	4.0	Complete
Wall collision	3.5	5.0	Complete
Waypoint detection and traversal	2.0	3.0	Complete
Game completion / score / loss reason	1.5	1.0	Complete
Disruptor / quantum tunnel	3.5	7.0	Complete
Tilt indication	1.0	0.5	Complete
Functional testing and debug	2.0	2.5	Complete
Week 4-5 documentation	1.0	2.0	Complete
TOTAL	42.5	45.0	

3. Single Work Item Evaluation

The disruptor tunnel task had the largest discrepancy: 3.5 hours estimated vs. 7.0 hours actual. Meaning a whole 2.0x over time budget.

The overrun came entirely from a hard fault on every button press. The GDB dump showed r4-r12 = 0xA5A5A5A5 (FreeRTOS stack fill pattern) and LR = 0xFFFFFFF1 (fault in handler mode during PendSV), confirming a stack overflow. The root cause was LEDTask's 256-byte stack. The Cortex-M4F FPU context save requires 204 bytes minimum per context switch, leaving almost nothing for the task itself. The fix was simply increasing LEDTask and GyroTask stacks to 1024 bytes each. This was not obvious from the just observing because the fault looked nothing like a stack overflow. The estimate assumed the button/disruptor logic would be the hard part, not the surrounding RTOS infrastructure.

4. Final Solution Analysis: RT Tasks

Task Priorities and Rationale

Three tasks run at osPriorityHigh: GyroTask, PhysicsTask, and ButtonTask. Three run at osPriorityNormal: LEDTask, LCDTask, and GameLogicTask.

GyroTask and PhysicsTask are high priority because they form the physics pipeline. Gyro data must be fresh when physics runs, and physics must complete within the 20 ms tick period to maintain consistent simulation timing. Stale gyro or delayed physics would produce jitter visible as drone lag. ButtonTask is high priority so the disruptor activation response is immediate. A user pressing the button to avoid a hole definitely shouldn’t wait behind an LCD redraw.

LEDTask, LCDTask, and GameLogicTask are normal priority. LED updates are cosmetic and can tolerate some latency. LCDTask runs at 100 ms (10 Hz) intentionally as full-

screen LCD operations are slow and would block the high-priority tasks if given higher priority. GameLogicTask is event-driven by g_physicsFlags and wakes only when physics completes, so its priority does not affect latency.

SystemView Analysis - Baseline (No Button)

The following captures show task scheduling during normal game play without button activation.



Figure 1. SystemView timeline - baseline operation. GyroTask and PhysicsTask fire periodically at 20 ms. LCDTask visible every 100 ms. Idle dominates.

Context

Name	Type	Core	Stack Information	Activations	Total Blocked Time	Total Run Time	Time Interrupted	CPU Load	Last Run Time	Min Run Time	Max Run Time	Min Blocked Time	Max Blocked Time	Run Time/s	Min Run Time/s
Scheduler		Core 0		0		0.000 000 000 s	0.000 000 ms	0.00 %	0.000 000 ms	0.000 000 ms	0.000 000 ms			0.000 000 ms	0.000 000 ms
Tmr Svc	@2	Core 0	209 @ 0x20026808	0	0.000 000 000 s	0.000 000 000 s	0.000 000 ms	0.00 %	0.000 000 ms	0.000 000 ms	0.000 000 ms	0.000 000 ms	0.000 000 ms	0.000 000 ms	0.000 000 ms
GameLogicTask	@24	Core 0	154 @ 0x200296E0	0	0.000 000 000 s	0.000 000 000 s	0.000 000 ms	0.00 %	0.000 000 ms	0.000 000 ms	0.000 000 ms	0.000 000 ms	0.000 000 ms	0.000 000 ms	0.000 000 ms
LCDTask	@24	Core 0	411 @ 0x20028E28	3	0.000 000 000 s	0.000 954 375 s	0.000 000 ms	0.04 %	0.316 750 ms	0.316 750 ms	0.318 875 ms	0.119 500 ms	0.119 625 ms	0.318 750 ms	0.318 750 ms
LEDTask	@24	Core 0	156 @ 0x20028970	128	0.088 968 250 s	0.089 540 750 s	0.000 000 ms	3.45 %	0.697 500 ms	0.290 250 ms	0.900 875 ms	0.481 875 ms	2.143 500 ms	34.287 125 ms	34.287 125 ms
defaultTask	@24	Core 0	89 @ 0x20027288	0	0.000 000 000 s	0.000 000 000 s	0.000 000 ms	0.00 %	0.000 000 ms	0.000 000 ms	0.000 000 ms	0.000 000 ms	0.000 000 ms	0.000 000 ms	0.000 000 ms
ButtonTask	@40	Core 0	200 @ 0x20028488	0	0.000 000 000 s	0.000 000 000 s	0.000 000 ms	0.00 %	0.000 000 ms	0.000 000 ms	0.000 000 ms	0.000 000 ms	0.000 000 ms	0.000 000 ms	0.000 000 ms
GyroTask	@40	Core 0	108 @ 0x20027748	128	0.000 000 000 s	0.200 242 125 s	0.000 000 ms	7.72 %	1.574 750 ms	0.713 625 ms	2.112 750 ms	0.115 000 ms	0.274 875 ms	77.655 875 ms	77.655 875 ms
PhysicsTask	@40	Core 0	404 @ 0x20027C00	129	0.001 605 750 s	0.090 526 375 s	0.000 000 ms	3.49 %	0.680 750 ms	0.537 250 ms	0.902 125 ms	0.115 375 ms	0.213 375 ms	34.991 875 ms	34.991 875 ms
Idle		Core 0		243		2.190 675 125 s	0.000 000 ms	84.44 %	0.000 000 ms	0.312 375 ms	17.955 000 ms			852.283 750 ms	852.283 750 ms

Figure 2. Context table - baseline. Idle = 84.4% CPU. PhysicsTask = 3.5%, GyroTask = 7.7%, LEDTask = 3.5%, LCDTask = 0.04%.

SystemView Analysis - With Disruptor Active

The following captures show the same game but with the user button held (the disruptor being active).



Figure 3. SystemView timeline with ButtonTask active. ButtonTask wakeup visible at center. PhysicsTask cycles are visibly wider due to added energy drain and quantum tunnel computation.

Events List Core 0

#	Time	Context	Event	Resource	Detail
4858	3.200 286 375	GyroTask	vTaskDelay		xTicksToDelay = 20
4859	3.200 583 375	Idle	System Idle		Idle for 2.476 ms
4860	3.202 940 625	Idle	Task Ready		ButtonTask, runs after 119.625 us
4861	3.203 060 250	ButtonTask	Task Run		Runs for 842.875 us
4862	3.203 218 250	ButtonTask	xQueueGenericReceive	0x1002...	xQueue = 0x100275F0 pvBuffer = 0xF0000000 xTicksToWait = 4294967295 xJustPeek = 1
4863	3.203 356 875	ButtonTask	xQueueGenericSend	0x1002...	xQueue = 0x100275F0 pvItemToQueue = 0x00000000 xTicksToWait = 0 xCopyPosition = 0
4864	3.203 903 125	Idle	System Idle		Idle for 10.156 ms
4865	3.213 940 375	Idle	Task Ready		PhysicsTask, runs after 119.625 us
4866	3.214 060 000	PhysicsTask	Task Run		Runs for 5.231 ms
4867	3.214 211 750	PhysicsTask	xQueueGenericReceive	0x1002...	xQueue = 0x10027648 pvBuffer = 0xF0000000 xTicksToWait = 4294967295 xJustPeek = 1
4868	3.214 349 500	PhysicsTask	xQueueGenericSend	0x1002...	xQueue = 0x10027648 pvItemToQueue = 0x00000000 xTicksToWait = 0 xCopyPosition = 0
4869	3.214 550 375	PhysicsTask	xQueueGenericReceive	0x1002...	xQueue = 0x10027540 pvBuffer = 0xF0000000 xTicksToWait = 4294967295 xJustPeek = 1
4870	3.214 690 500	PhysicsTask	xQueueGenericSend	0x1002...	xQueue = 0x10027540 pvItemToQueue = 0x00000000 xTicksToWait = 0 xCopyPosition = 0
4871	3.214 861 250	PhysicsTask	xQueueGenericReceive	0x1002...	xQueue = 0x10027598 pvBuffer = 0xF0000000 xTicksToWait = 4294967295 xJustPeek = 1
4872	3.215 028 875	PhysicsTask	xQueueGenericSend	0x1002...	xQueue = 0x10027598 pvItemToQueue = 0x00000000 xTicksToWait = 0 xCopyPosition = 0
4873	3.215 198 375	PhysicsTask	xQueueGenericReceive	0x1002...	xQueue = 0x100275F0 pvBuffer = 0xF0000000 xTicksToWait = 4294967295 xJustPeek = 1
4874	3.215 338 125	PhysicsTask	xQueueGenericSend	0x1002...	xQueue = 0x100275F0 pvItemToQueue = 0x00000000 xTicksToWait = 0 xCopyPosition = 0
4875	3.215 940 375	PhysicsTask	Task Ready		LEDTask, runs after 3.350 ms
4876	3.218 221 125	PhysicsTask	xQueueGenericReceive	0x1002...	xQueue = 0x10027598 pvBuffer = 0xF0000000 xTicksToWait = 4294967295 xJustPeek = 1
4877	3.218 366 375	PhysicsTask	xQueueGenericSend	0x1002...	xQueue = 0x10027598 pvItemToQueue = 0x00000000 xTicksToWait = 0 xCopyPosition = 0
4878	3.218 563 375	PhysicsTask	xQueueGenericReceive	0x1002...	xQueue = 0x100275F0 pvBuffer = 0xF0000000 xTicksToWait = 4294967295 xJustPeek = 1
4879	3.218 710 125	PhysicsTask	xQueueGenericSend	0x1002...	xQueue = 0x100275F0 pvItemToQueue = 0x00000000 xTicksToWait = 0 xCopyPosition = 0
4880	3.218 957 625	PhysicsTask	Task Ready		GameLogicTask, runs after 926.500 us
4881	3.219 081 750	PhysicsTask	vTaskDelay		xTicksToDelay = 20
4882	3.219 291 250	LEDTask	Task Run		Runs for 592.875 us
4883	3.219 443 625	LEDTask	xQueueGenericReceive	0x1002...	xQueue = 0x100275F0 pvBuffer = 0xF0000000 xTicksToWait = 4294967295 xJustPeek = 1
4884	3.219 612 875	LEDTask	xQueueGenericSend	0x1002...	xQueue = 0x100275F0 pvItemToQueue = 0x00000000 xTicksToWait = 0 xCopyPosition = 0
4885	3.219 731 750	LEDTask	vTaskDelay		xTicksToDelay = 20
4886	3.219 884 125	GameLogicTask	Task Run		Runs for 219.375 us
4887	3.219 986 125	GameLogicTask	Task Ready		GyroTask, runs after 117.375 us
4888	3.220 103 500	GyroTask	Task Run		Runs for 1.495 ms
4889	3.221 116 375	GyroTask	xQueueGenericReceive	0x1002...	xQueue = 0x10027540 pvBuffer = 0xF0000000 xTicksToWait = 4294967295 xJustPeek = 1
4890	3.221 258 375	GyroTask	xQueueGenericSend	0x1002...	xQueue = 0x10027540 pvItemToQueue = 0x00000000 xTicksToWait = 0 xCopyPosition = 0
4891	3.221 355 875	GyroTask	vTaskDelay		xTicksToDelay = 20
4892	3.221 599 250	GameLogicTask	Task Run		Runs for 991.875 us
4893	3.221 764 375	GameLogicTask	xQueueGenericReceive	0x1002...	xQueue = 0x10027598 pvBuffer = 0xF0000000 xTicksToWait = 4294967295 xJustPeek = 1
4894	3.221 903 000	GameLogicTask	xQueueGenericSend	0x1002...	xQueue = 0x10027598 pvItemToQueue = 0x00000000 xTicksToWait = 0 xCopyPosition = 0
4895	3.222 114 375	GameLogicTask	xQueueGenericReceive	0x1002...	xQueue = 0x10027648 pvBuffer = 0xF0000000 xTicksToWait = 4294967295 xJustPeek = 1
4896	3.222 253 000	GameLogicTask	xQueueGenericSend	0x1002...	xQueue = 0x10027648 pvItemToQueue = 0x00000000 xTicksToWait = 0 xCopyPosition = 0
4897	3.222 591 125	Idle	System Idle		Idle for 16.561 ms
4898	3.238 940 500	Idle	Task Ready		PhysicsTask, runs after 212.250 us

Figure 4. Context table with Disruptor active. PhysicsTask jumps to 19.2% CPU (from 3.5%), Idle drops to 60.2% (from 84.4%). LCDTask increases to 5.3% due to more frequent drone redraws during tunneling. ButtonTask shows 0.07%.

Observed Timing Characteristics

The most obvious consumers with the disruptor active are PhysicsTask (19.2%) and GyroTask (7.6%). The PhysicsTask increase is explained by two factors: wall collision is skipped during normal play when the drone is not near a wall, but Physics_QuantumTunnel runs a 200-step loop over all 225 possible wall segments on deactivation. LCDTask increases from

0.04% to 5.3% because LCD_RestoreNearDrone redraws walls and holes at both the old and new drone positions every frame during pass-through.

LCDTask shows a max blocked time of approximately 10 ms in the context table, which is expected - it calls LCD_Draw_Circle_Fill and per-pixel wall drawing routines that are inherently slow on the ILI9341 over SPI. This is why LCDTask was given Normal priority and a 100 ms period rather than the 20 ms physics period.

I didn't see any timing violations and all tasks completed within their periods. The system has approximately 60% headroom at idle during worst-case disruptor operation.

5. Final Solution Analysis: Code Space

The build output (arm-none-eabi-size) for the final binary reports: text = 68,468 bytes, data = 124 bytes, bss = 180,204 bytes, total = 248,796 bytes. The STM32F429ZI has 2 MB of flash and 256 KB of SRAM (plus 64 KB CCM RAM). The relevant memory segments are:

Segment	Approximate Size	Location	Contents
.text (flash)	66.9 KB (68,468 bytes)	Flash	All compiled code: application, FreeRTOS kernel, HAL drivers, LCD driver, math library (sinf)
.data (flash + RAM)	124 bytes	Copied to RAM at startup	Initialized globals: Config struct, initial task attribute structs
.bss (RAM)	176.0 KB (180,204 bytes)	SRAM 0x20000000	Uninit globals: frameBuffer[240x320] = 153,600 bytes (85.2% of BSS); FreeRTOS heap = 15,360 bytes; maze arrays (wallsH, wallsV, holeCenters) ~2 KB; task state structs ~0.5 KB
FreeRTOS heap	15 KB	Within .bss	Task stacks (12 KB across 7 tasks), TCBs, mutexes, semaphores, event flags

The largest RAM consumer is the LCD framebuffer at 153,600 bytes. About 85.2% of BSS and 58.6% of total SRAM. Flash usage is just 68,592 bytes out of 2,097,152 (3.3%) which is well within budget. RAM usage is 180,328 of 262,144 bytes (68.8%). If the framebuffer were moved to the STM32F429I-DISC1's 8 MB external SDRAM, RAM usage would drop to 26,728 bytes (10.2%), which is the first change a production version would make.

The FreeRTOS heap is 15 KB in which approximately 11.8 KB is allocated at runtime for all task stacks, TCBs, and ITC objects, leaving just 3.5 KB margin. Stack sizes were set conservatively after diagnosing the LEDTask overflow, the Cortex-M4F FPU context save requirement (204 bytes minimum per context switch) is the binding constraint, not the task logic itself.

The text segment at 66.9 KB breaks down roughly as: HAL driver library ~25 KB, FreeRTOS kernel ~15 KB, LCD driver with font tables ~12 KB, application code (ApplicationCode.c) ~10 KB, and math library (sinf) ~5 KB. The data segment is only 124 bytes

which is the Config struct initializer and a handful of other initialized globals. The small data size reflects the deliberate choice to keep Config as a const struct, which the compiler places in flash (text) rather than RAM.

6. Final Solution Analysis: Physics Update

How the Physics Was Divided

The physics update is split across two pure helper functions called from PhysicsTask each 20 ms cycle, plus inline collision and bounds logic within the task itself.

Physics_UpdateVelocity() handles the dynamics: it takes the current velocity and board angles, applies AngleGain scaling, converts to radians, projects gravity onto X and Y through sinf(), and integrates acceleration over DELTA_T_S. This function has no RTOS calls, no maze knowledge, and no side effects - it is a pure transform of inputs to outputs.

Physics_CheckHoleFallIn() is similarly pure: it takes drone position, hole center arrays, hole diameter, and the disruptor flag, and returns a boolean. It has no knowledge of walls, velocity, or game state.

Wall collision, border clamping, disruptor energy drain, OOB tilt detection, and quantum tunnel are all inline within PhysicsTask, operating on local copies of state read under mutex at the top of each cycle.

Why This Division

The separation of dynamics from collision was a deliberate testability decision. Physics_UpdateVelocity and Physics_CheckHoleFallIn are functions with no dependencies on FreeRTOS or the HAL. They can be compiled and unit tested on a host machine with a simple test harness, with analytically verified expected outputs (5 degree tilt -> 854 mm/s^2 acceleration -> 17.1 mm/s after one 20 ms tick). The two unit tests in the functional test plan exercise exactly this.

Keeping wall collision inline in PhysicsTask rather than a separate function was just a simplicity choice. Wall collision modifies posX, posY, velX, velY simultaneously and needs access to the full wallsH and wallsV arrays. Passing all of that through a function signature would have added complexity since the maze arrays are global and the function would still have side effects through its pointer arguments.

What Caused Extra Work

The main unexpected complexity in the physics pipeline was the collision radius tuning. The initial implementation used DRONE_RADIUS_MM (5 mm) as the push-back distance. In a 12 mm cell with walls on both sides, that leaves exactly 1 mm of navigable clearance for the drone center. Technically it should be passable but nearly it’s impossible in practice because any off-axis approach sends the drone into the wall. Reducing to DRONE_RADIUS_MM - 1.0f (4 mm effective radius) gave 3 mm of clearance and made single-cell corridors navigable.

The second unexpected issue was wall endpoint behavior. An early version extended the wall extent check by DRONE_RADIUS_MM on each end to catch near-endpoint collisions. This caused phantom collisions at wall ends. The drone would hit an invisible barrier in a gap

between two non-adjacent wall segments. Removing the extent extension entirely and using exact cell boundaries fixed this, at the cost of allowing the drone to clip wall endpoints. But I consider this visually acceptable since its only about one pixel per mm.

7. Final Solution Analysis: Configuration Data

The default AngleGain of 500 maps 90 degrees of physical tilt to 45 degrees of simulated tilt. This was well-chosen for playability as the ball moves fast enough to feel responsive but not so fast that it is uncontrollable. Gravity at 980 cm/s^2 combined with AngleGain 500 produces a maximum acceleration of 9800 * sin(45) = 6930 mm/s^2 at full tilt, which gives the drone a satisfying physics feel without being instant.

VelocityDamping at 0.98 per cycle was essential. Without damping, the drone would oscillate against walls indefinitely and any energy absorbed on impact would be immediately re-added on the next cycle. At 50 Hz, 0.98 per cycle means velocity halves in about 34 cycles (0.68 seconds), which I personally think feels like a light rolling friction.

The wall and hole probabilities (10% and 5% respectively) had to be reduced from the given suggested values. Higher wall probability produced mazes that were effectively unsolvable because too many cells were blocked by walls on multiple sides. At 10%, the maze has enough walls to be interesting without being unreachable. At 5%, holes are frequent enough to require attention but not so dense that avoidance dominates the game.

The 30-second time limit can be tight sometimes but is still very achievable. A good run can complete all three waypoints in under 10 seconds.

Config values that were unnecessary or redundant: PinAtCenter is required to be 0 (MAZE mode) and is validated at startup, but there is no implementation for PinAtCenter=DRONE mode. The field serves as a mode selector that currently has only one valid value. Waypoint Diameter (90 mm, 45 mm radius) was specified in the config but the effective detection radius was reduced to 8 mm in code to match the visual dot size. The config value is not actually used for collision detection, only for drawing. This is a config-to-code discrepancy that I would probably clean up in a newer version.

8. If I Had 2 More Weeks

I think that the best-value addition would be dynamic hazards that change during gameplay. Currently the maze is static after generation with the walls and holes never moving. Adding closing walls, moving holes, or a shrinking boundary would significantly increase difficulty and replayability without requiring huge changes to the core physics.

This is the right choice because playability is the primary design goal, and the current maze becomes predictable once the player sees the layout. Dynamic hazards would keep every run different even on the same generated maze. The implementation is bounded with the wallsH and wallsV arrays already read at each physics cycle for collision. So toggling a wall on or off is a simple single byte write. A simple timer-driven wall activation task could be added as a seventh RTOS task with minimal impact given the remaining 3.5 KB margin.

The second addition would be a pre-game config override menu. The current Config struct is hardcoded and requires a reflash to change any value. A simple two-button (tilt left/right to navigate, button to select) menu on the LCD at startup would allow the professor to change the number of waypoints, maze dimensions, or hole density without touching the code. This is a good choice since I think it would be achievable in about 4-5 hours given that all the LCD drawing infrastructure already exists.